

AI 创作平台

项目经验与技术实现

基于 LangGraph 多 Agent 协作的智能小说创作系统

项目类型	全栈 Web 应用 — AI 辅助创意写作平台
技术栈	Vue 3 + TypeScript / FastAPI + SQLAlchemy / LangGraph / PostgreSQL
核心能力	多 Agent 协作写作、RAG 上下文检索、实时流式输出、人机协同审批
开发方式	CCB 多 Agent 协同开发 (Claude Code Bridge 多实例并行编码)

技术关键词

LangGraph SSE 流式 RAG 多Agent Vue 3 FastAPI Pinia PostgreSQL JWT Docker

一、项目概述

AI 创作平台是一个面向小说创作者的智能辅助写作系统。用户可以创建写作项目，构建大纲、角色、世界观等创作素材，然后通过多种 AI 协作模式辅助创作：章节生成（全流水线）、润色优化、转折建议续写、读者视角反馈。系统采用多 Agent 架构，每个 Agent 扮演不同角色（创意大师、审校大师、一致性检查等），通过 LangGraph 状态图编排协作流程，支持并行执行和人工审批。

1.1 核心业务场景

章节生成：用户选择大纲/角色/世界观素材，设定目标字数，AI 走完整流水线生成章节内容

润色优化：AI 先分析章节给出 3 个润色方向，用户选择后按方向优化全文

转折建议：AI 分析当前写作进度给出 5 个方向建议，用户选择后逐字续写到编辑器

读者视角：AI 以读者身份给出阅读体验反馈，不修改原文

1.2 技术架构总览

系统采用前后端分离架构。前端 Vue 3 + TypeScript + Pinia 状态管理，后端 FastAPI + SQLAlchemy 异步 ORM + PostgreSQL，AI 工作流基于 LangGraph StateGraph，实时通信使用 SSE (Server-Sent Events) 流式推送。部署采用 Docker Compose 容器化方案。

表现层	Vue 3 + TypeScript + Pinia + Vue Router，响应式组件化 UI
通信层	SSE 流式推送（writer_output token 级流式），REST API CRUD
业务层	FastAPI 路由 + LangGraph StateGraph 多 Agent 工作流编排
AI 层	LLM Provider 抽象（OpenAI 兼容协议），RAG 向量检索上下文注入
数据层	PostgreSQL + SQLAlchemy async ORM + Alembic 迁移 + pgvector
安全层	JWT 认证 + bcrypt 密码哈希 + Fernet 对称加密 + Token Bucket 限流

二、开发方式：CCB 多 Agent 协同

本项目采用 CCB (Claude Code Bridge) 多 Agent 协同开发模式。CCB 是一个基于 Claude Code CLI 的多实例并行编码框架，通过共享项目上下文和任务分发机制，实现多个 AI Agent 同时开发不同模块。

2.1 CCB 工作原理

CCB 在一个项目中配置多个 Agent 实例（如 commander、worker1、worker2），每个 Agent 拥有独立的 Claude Code 会话和工作目录。commander 负责任务拆解和分发，worker 负责具体实现。Agent 之间通过 /ask 命令进行异步消息通信，共享项目记忆（ccb_memory.md）保持上下文一致。

Agent 角色	commander（任务调度）+ worker1/worker2（并行实现）
通信方式	/ask <agent> <message> 异步消息，支持广播 (/ask all)
共享记忆	ccb_memory.md — 项目级知识库，所有 Agent 可读写
隔离机制	每个 Agent 独立 git worktree，避免代码冲突
任务追踪	自动生成任务列表，commander 追踪完成状态

2.2 实际协作流程

- commander 分析需求，拆解为前端/后端独立任务
- 通过 /ask worker1 /ask worker2 分发任务，附带文件路径和实现要求
- worker 在独立 worktree 中实现，完成后回复 commander
- commander 审查代码，合并变更，处理冲突
- 共享记忆确保每个 Agent 了解项目最新状态和设计决策

2.3 开发效率提升

相比单 Agent 串行开发，CCB 多 Agent 并行模式显著提升了开发效率：前后端任务可同时推进，复杂重构可拆分到不同 Agent 并行处理。在实际开发中，一个典型的三模式重构（章节生成/润色/转折建议）从需求分析到前后端联调完成，CCB 并行模式比串行模式节省约 40% 的时间。同时，Agent 之间的代码审查机制也提高了代码质量。

三、后端架构与模块详解

3.1 API 路由层 (api/routes.py)

FastAPI 路由层是整个后端的入口，负责 HTTP 请求处理、参数校验和 SSE 流式响应。核心端点是 `POST /api/projects/{id}/generate`，根据 `mode` 参数分流到不同的生成逻辑。

<code>POST /generate</code>	核心生成端点，支持 4 种模式分流 + SSE 流式输出
<code>POST /auth/register</code>	用户注册，bcrypt 哈希密码
<code>POST /auth/login</code>	用户登录，返回 JWT access_token
<code>CRUD /projects</code>	项目管理（创建/列表/详情/更新/删除）
<code>CRUD /chapters</code>	章节管理，支持排序和内容更新
<code>CRUD /outlines</code>	大纲管理，含 <code>sequence_number</code> 排序
<code>CRUD /characters</code>	角色管理，含 <code>role_type</code> 分类
<code>CRUD /world-entries</code>	世界观词条管理，含 <code>category</code> 分类
<code>CRUD /experts</code>	专家 Agent 配置管理
<code>POST /experts/{id}/test</code>	专家测试流端点，SSE 流式输出
<code>GET/PUT /llm-settings</code>	用户 LLM 配置（API Key 加密存储）

生成模式分流逻辑

`generate` 端点根据 `mode` 字段执行不同流程：

<code>full_pipeline</code>	走 LangGraph 完整流水线： <code>context_loader</code> → <code>writer</code> → [<code>critic</code> // <code>consistency</code>] → <code>human_review</code>
<code>enhance</code> （无方向）	LLM 分析章节内容，输出 3 个润色方向建议（SSE: <code>enhance_directions</code> ）
<code>enhance</code> （有方向）	按用户选定方向润色全文，流式输出 <code>token</code> ，替换章节内容
<code>continue</code> （无方向）	LLM 分析写作进度，输出 5 个转折建议（SSE: <code>turn_suggestions</code> ）
<code>continue</code> （有方向）	按选定方向续写，流式 <code>token</code> 追加到章节末尾
<code>summarize</code>	读者视角反馈，单次 LLM 调用，不修改原文

3.2 LangGraph 多 Agent 工作流 (agents/workflow.py)

工作流是系统的核心创新点。使用 `LangGraph` `StateGraph` 编排多个 `AI Agent` 的协作流程，支持动态图构建、并行执行、人工审批和 Checkpoint 持久化。

状态定义 — CreativeState

所有节点共享一个 `TypedDict` 状态对象，包含项目 ID、上下文、草稿、审校意见、一致性报告、修订次数等字段。`critiques` 字段使用 `Annotated[list, reducer]` 实现意见累积。

默认流水线

默认流水线为：context_loader → writer → [critic // consistency_checker] → human_review。critic 和 consistency_checker 从 writer 后并行执行，通过 LangGraph 的并行边机制实现，两者都汇入 human_review 节点。这比串行执行节省约 50% 的审校时间。

动态图构建

系统支持用户自定义专家 Agent，通过 Expert 模型的 workflow_position 字段 (replace_writer / replace_critic / pre_writer / post_writer / pre_critic / post_critic / standalone) 动态构建图节点和边。_make_expert_node 工厂函数为每个自定义专家生成节点函数，支持 writer / critic / editor / researcher 四种角色类型。

人工审批 (HITL)

human_review 节点通过 interrupt_before 机制暂停 workflow，等待用户决策 (接受/修改后接受/拒绝)。配合 MemorySaver checkpoint，支持断点续作和状态恢复。route_after_review 根据修订次数决定是否继续迭代。

3.3 LM Provider 抽象层 (agents/llm_provider.py)

LLM Provider 是 AI 调用的统一抽象层，支持 mock / openai / deepseek / siliconflow / zhipu / moonshot / qwen / yi / minimax / custom 等厂商。所有非 mock 厂商均走 OpenAI 兼容 API (AsyncOpenAI 客户端)，只需配置 api_key + base_url + model 即可接入新厂商。

MockProvider	测试用，返回上下文感知的模拟响应 (提取角色名/大纲标题)
OpenAIProvider	生产用，支持 generate (同步) 和 generate_stream (流式 token)
get_llm_provider()	工厂函数，根据用户 LLM 配置或 settings 默认值返回实例

用户可在前端设置页面配置自己的 LLM 厂商和 API Key，API Key 使用 Fernet 对称加密存储在数据库中，运行时解密传入 Provider。这使得每个用户可以使用不同的 AI 模型，互不影响。

3.4 RAG 上下文检索 (rag/)

RAG 模块负责为 AI 生成提供创作上下文，包含两条路径：

向量搜索路径：ContextLoader 通过 embedder 向量化素材，vector_store 做相似度检索，chunker 负责文本分块，indexer 管理索引构建。适用于无明确素材选择时的智能推荐

精确加载路径：当用户在 ContextPicker 中主动选择大纲/角色/世界观 ID 时，context_loader_node 按 ID 精确加载，不截断不走向量搜索，确保用户选中的素材完整注入

context_loader.py	上下文加载入口，支持向量搜索和 ID 精确加载两种模式
embedder.py	文本向量化，调用 LLM embedding API
vector_store.py	向量存储与相似度检索 (pgvector)
chunker.py	文本分块策略，按段落/字数切分
indexer.py	索引构建与管理，素材变更时自动更新向量索引

3.5 数据模型层 (models/)

使用 SQLAlchemy 2.0 声明式映射 + async session，所有模型继承自 Base (TimestampMixin)。通过 Alembic 管理数据库迁移。

User	用户表, bcrypt 密码哈希, JWT 认证
Project	项目表, 关联用户, 含 genre/setting 等创作元数据
Chapter	章节表, 含 sequence_number 排序、content 正文、word_count 字数统计
Outline	大纲表, 含 sequence_number、title、summary、turning_point
Character	角色表, 含 role_type (主角/配角/反派)、profile 角色简介
CharacterRelation	角色关系表, 多对多关联, 含 relation_type 描述
WorldEntry	世界观词条表, 含 category 分类和 content 详细设定
HiddenThread	暗线表, 用于管理故事中的隐藏线索和伏笔
Expert	专家 Agent 配置表, 含 role_type、system_prompt、workflow_position、temperature、max_tokens
LLMConfig	用户 LLM 配置表, api_key 使用 Fernet 加密存储

3.6 认证与安全模块

JWT 认证 (api/auth.py)：注册时 bcrypt 哈希密码，登录签发 JWT token，后续请求通过 Authorization header 验证身份

API Key 加密 (utils/crypto.py)：用户配置的 LLM API Key 使用 Fernet 对称加密存储，密钥从环境变量读取，运行时解密传入 Provider

Token Bucket 限流 (api/rate_limiter.py)：基于令牌桶算法的请求限流，防止 API 滥用，保护 LLM 调用配额

内容安全 (agents/safety.py)：AI 输出内容安全检查，过滤敏感内容

四、前端架构与模块详解

4.1 技术选型与工程配置

前端基于 Vue 3 Composition API + TypeScript，构建工具使用 Vite。状态管理采用 Pinia（组合式 API 风格），路由使用 Vue Router 4。CSS 使用自定义属性（CSS Variables）设计系统，支持暗色模式。

框架	Vue 3.4+ Composition API + <script setup>
语言	TypeScript 严格模式
构建	Vite 5, HMR 热更新
状态管理	Pinia 组合式 Store（useChapterStore / useExpertStore / useUiStore 等）
路由	Vue Router 4, 动态路由 /projects/:id/workspace
HTTP	原生 fetch + SSE EventSource, 封装 api/client.ts
样式	CSS Variables 设计系统, scoped 样式, 暗色模式支持

4.2 页面与组件体系

核心页面（views/）

Login.vue	登录/注册页面, JWT 认证流程
Dashboard.vue	项目列表仪表盘, 创建/删除项目
Workspace.vue	核心工作区, 三栏布局: 素材面板 + 编辑器 + Agent 面板
AgentStudio.vue	Agent 配置工作室, 创建/编辑/测试自定义专家
Settings.vue	系统设置, LLM 厂商配置 + API Key 管理

核心组件（components/）

NovelEditor.vue	小说编辑器, 实时编辑章节内容, 支持 AI 续写追加
AgentPanel.vue	Agent 协作面板, 三种模式入口 + SSE 事件处理 + workflow 展示
AgentWorkflow.vue	竖向 workflow 进度条, 5 节点 (context/writer/critic/consistency/review)
AgentCreator.vue	专家创建/编辑表单, 含角色类型/workflow 位置/温度/Token 配置
ContextPicker.vue	素材选择弹窗, 大纲/角色/世界观多选 + 目标字数输入
EnhancePicker.vue	润色方向选择弹窗, AI 给出 3 方向 + 用户补充说明
TurnPicker.vue	转折方向选择弹窗, AI 给出 5 建议 + 用户补充说明
ApprovalModal.vue	人工审批弹窗, 接受/修改后接受/拒绝

4.3 Pinia 状态管理

前端使用多个 Pinia 组合式 Store 管理应用状态，各 Store 职责清晰、互不耦合。

useChapterStore	章节 CRUD、当前章节选择、草稿更新（updateDraft）、自动保存
useExpertStore	专家列表、生成状态（isGenerating）、工作流步骤（workflowSteps）、流式输出（streamOutput）、最终草稿（finalDraft/appendDraft/replaceDraft）
useOutlineStore	大纲 CRUD、按项目分组
useCharacterStore	角色 CRUD、按项目分组
useWorldEntryStore	世界观词条 CRUD、按项目分组
useUiStore	全局 UI 状态、Toast 通知、模态框管理
useLlmSettingsStore	用户 LLM 配置、API Key 管理

4.4 SSE 流式通信机制

前端通过 `api/client.ts` 封装的 `generateStream` 方法与后端建立 SSE 连接，接收多种事件类型的流式数据。这是实现 AI 逐字输出、实时进度展示和两步交互（方向选择）的关键机制。

progress	系统进度消息，appendOutput 显示在输出区
agent_start	Agent 开始执行，更新工作流步骤状态为 running
agent_output	Agent 中间输出 token，appendOutput 显示
writer_output	Writer 生成 token，根据模式分流：continue→编辑器 / enhance→finalDraft / full_pipeline→finalDraft
critic_output	审校意见，appendOutput 显示，不写入 finalDraft
consistency_check	一致性报告，appendOutput 显示，不写入 finalDraft
enhance_directions	润色方向建议（3 个），弹出 EnhancePicker 供用户选择
turn_suggestions	转折方向建议（5 个），弹出 TurnPicker 供用户选择
done	生成完成，full_pipeline 弹审批弹窗 / enhance 自动应用 / continue 已在编辑器
error	错误消息，Toast 提示 + appendOutput

模式差异化处理

同一 `writer_output` 事件在不同模式下有完全不同的处理逻辑：continue 模式将 token 直接追加到 NovelEditor 的草稿末尾（用户实时看到续写），enhance 模式将 token 累积到 `expertStore.finalDraft`（完成后一次性替换全文），`full_pipeline` 模式同样累积到 `finalDraft`，完成后弹出审批弹窗。这种设计确保了三种模式的用户体验差异：续写是实时追加、润色是整体替换、生成是审批后应用。

4.5 CSS 设计系统

前端使用 CSS Custom Properties 构建了一套完整的设计系统，包含间距（`--sp-1 ~ --sp-8`）、字号（`--text-xs ~ --text-xl`）、颜色（`--accent / --bg-panel / --border` 等）、圆角（`--radius / --radius-lg`）、过渡（`--transition`）等语义化变量。所有组件使用 `scoped` 样式 +

变量引用，确保视觉一致性。暗色模式通过覆盖 CSS 变量实现，无需额外 class 切换。

五、核心设计决策与技术亮点

5.1 三模式差异化交互

系统将 AI 辅助创作拆分为三种独立模式，每种有不同的交互流程、后端逻辑和输出方式：

章节生成	ContextPicker 选素材+字数 → LangGraph 全流水线 → 审批弹窗 → 应用纯小说内容
润色优化	AI 分析 → 3 方向建议 → 用户选择+补充 → 按方向润色 → 替换全文
转折续写	AI 分析 → 5 方向建议 → 用户选择+补充 → 逐字追加到编辑器末尾

关键设计：润色和续写的两步交互（先给方向再执行）避免了 AI 盲目输出不符合用户意图的内容，大幅提升了生成结果的可用性。续写模式的 token 直接追加到编辑器，让用户实时看到 AI 写作过程，体验更自然。

5.2 并行审校优化

在 LangGraph 流水线中，critic（审校）和 consistency_checker（一致性检查）从 writer 后并行执行，而非串行。这两个节点的输入都是 writer 的输出，互不依赖，并行执行将审校阶段耗时缩短约 50%。LangGraph 原生支持并行边（graph.add_edge("writer", "critic") + graph.add_edge("writer", "consistency_checker"）），两个分支都汇入 human_review 节点后继续。

5.3 输出纯净性保障

章节生成模式下，AI 流水线会产出审校意见、一致性报告等中间结果。设计上严格区分 finalDraft（纯小说内容）和 streamOutput（所有日志），writer_output 事件写入 finalDraft，critic_output / consistency_check 只写入 streamOutput。审批弹窗只展示 finalDraft，确保用户看到的始终是纯净的小说文本。这一设计避免了一个严重 bug：早期版本中审批弹窗可能为空，用户误点接受会用空内容覆盖原有文章。

5.4 LLM Provider 多厂商适配

通过 OpenAI 兼容协议统一接入所有 LLM 厂商。每个厂商只需配置 api_key + base_url + model 三个参数，底层统一使用 AsyncOpenAI 客户端。用户可在设置页面配置自己的 LLM 厂商，API Key 加密存储，生成时从数据库读取用户配置传入 get_llm_provider() 工厂函数。这种设计使得系统可以灵活切换 DeepSeek / 通义千问 / 智谱 / Moonshot 等国产模型，也可以使用 OpenAI / Claude 等国际模型。

5.5 上下文注入双路径

context_loader_node 支持两种上下文加载路径：当用户在 ContextPicker 中主动选择素材 ID 时，按 ID 精确加载，不截断不走向量搜索；当无选中 ID 时，走 RAG 向量搜索逻辑，智能推荐相关素材。精确路径确保用户选中的素材完整注入（大纲全文、角色完整简介、世界观完整设定），向量路径则提供智能化的上下文补充。

5.6 动态工作流图构建

`build_creative_graph()` 函数根据项目启用的 Expert 列表动态构建 LangGraph 图。Expert 的 `workflow_position` 字段决定其在流水线中的位置：`replace_writer/replace_critic` 替换默认节点，`pre_writer/post_writer/pre_critic/post_critic` 在默认节点前后插入自定义节点，`standalone` 独立运行。`_make_expert_node` 工厂函数为每个专家生成节点函数，支持自定义 `system_prompt`、`temperature`、`max_tokens`。这使得非技术用户也能通过 UI 配置自己的 AI Agent 团队。

六、数据流与交互流程

6.1 章节生成完整流程

1. 用户点击「章节生成」→ 弹出 ContextPicker
2. 用户选择大纲/角色/世界观素材 + 设定目标字数 → 确认
3. 前端调用 POST /generate (mode=full_pipeline) 建立 SSE 连接
4. 后端 context_loader_node 按 ID 精确加载选中素材
5. writer_node 根据上下文 + 目标字数生成章节内容（流式 token）
6. critic_node 和 consistency_checker_node 并行审校
7. human_review 节点 interrupt_before 暂停，等待用户决策
8. 前端弹出 ApprovalModal 展示纯小说内容
9. 用户接受 → 内容写入章节编辑器 → 自动保存

6.2 润色两步交互流程

1. 用户点击「润色」→ 前端调用 POST /generate (mode=enhance, 无 enhance_direction)
2. 后端 LLM 分析章节内容，输出 3 个润色方向 → SSE event: enhance_directions
3. 前端弹出 EnhancePicker 展示方向列表
4. 用户选择方向 + 填写补充说明 → 确认
5. 前端再次调用 POST /generate (mode=enhance, enhance_direction=选定方向, user_note=补充说明)
6. 后端按选定方向润色全文 → 流式输出 writer_output token
7. 前端累积 token 到 finalDraft → 完成后自动替换编辑器内容

6.3 转折续写两步交互流程

1. 用户点击「转折建议」→ 前端调用 POST /generate (mode=continue, 无 turn_direction)
2. 后端 LLM 分析写作进度，输出 5 个方向建议 → SSE event: turn_suggestions
3. 前端弹出 TurnPicker 展示建议列表
4. 用户选择方向 + 填写补充说明 → 确认
5. 前端再次调用 POST /generate (mode=continue, turn_direction=选定方向, user_note=补充说明)
6. 后端按选定方向续写 → 流式输出 writer_output token
7. 前端将 token 直接追加到编辑器当前章节末尾（用户实时看到续写）

七、部署与运维

7.1 容器化部署

项目使用 Docker Compose 编排前后端 + PostgreSQL 三个服务。后端 Dockerfile 基于 Python 3.11 slim 镜像，前端使用 Node 18 构建 + Nginx 托管静态文件。数据库使用 PostgreSQL 官方镜像 + pgvector 扩展支持向量检索。

后端服务	FastAPI + Uvicorn，端口 8000，挂载 /app 目录
前端服务	Nginx 托管构建产物，代理 /api 到后端
数据库	PostgreSQL 15 + pgvector，持久化数据卷
环境变量	DATABASE_URL / LLM_API_KEY / LLM_PROVIDER / FERNET_KEY / JWT_SECRET

7.2 数据库迁移

使用 Alembic 管理数据库 schema 迁移。每次模型变更后执行 alembic revision --autogenerate 生成迁移脚本，alembic upgrade head 应用迁移。迁移脚本纳入版本控制，确保多环境 schema 一致。

八、项目收获与反思

8.1 技术收获

- 深入理解 LangGraph 状态图编排，掌握了并行节点、条件路由、人工审批等高级特性
- 实践了 SSE 流式通信的完整方案，包括多事件类型、token 级流式、前端差异化处理
- 设计了 LLM Provider 抽象层，通过 OpenAI 兼容协议实现多厂商无缝切换
- 实现了 RAG 向量检索与精确加载的双路径上下文注入方案
- 掌握了 CCB 多 Agent 协同开发模式，提升了复杂项目的并行开发效率

8.2 踩坑与解决

- 空审批弹窗覆盖文章：done 事件对所有模式都弹审批弹窗，但 enhance/continue 模式下 finalDraft 为空，用户误点接受会用空内容覆盖原文。解决：仅 full_pipeline 弹审批，其他模式自动应用或跳过
- 续写内容输出到错误位置：writer_output 统一写入 expertStore，但 continue 模式需要写入编辑器。解决：根据 pendingMode 分流，continue 模式 token 直接追加到 chapterStore.draft
- MockProvider 忽略上下文：早期 MockProvider 返回硬编码文本，不反映用户选中的素材。解决：重写为上下文感知模式，从 user_prompt 中提取角色名/大纲标题/世界观并反映在输出中
- ContextPicker 选择 bug：label + v-model + display:flex 导致点击区域异常。解决：改用 div + @click + :checked 显式控制

8.3 未来优化方向

- 引入流式编辑器（如 Tiptap），支持 AI 续写的 diff 高亮和逐字动画效果
- 增加 Agent 记忆机制，让 AI 记住用户的写作风格偏好和修改习惯
- 支持多轮对话式创作，用户可以与 Agent 反复讨论和调整
- 引入 A/B 测试机制，让用户对比不同 Agent 配置的生成效果
- 优化 RAG 检索质量，引入重排序（reranker）和混合检索（向量+关键词）

本文档基于 AI 创作平台项目实际代码生成，涵盖架构设计、模块实现、核心决策和开发方法论。